Mittuniversitetet

MID SWEDEN UNIVERSITY

Laboration 4 - MQTT

DT171G, Inbäddade system för Internet of Things

Omar Sultanzoy (omsu2400)

Siye Abera (siab2400)

Wichitsin Sinonngam (wisi2403)

15 Februari 2026

Innehåll

1 Sammanfattning **2**

2 MQTT: Teori **3**

3 Hårdvarukrav **3**

4 Kretskopplings **4**
4.1 Pin-konfiguration 4
4.2 Mottagarkrets (LED) 4
4.3 Sändarkrets (Tryckknapp) 4

5 Fjärranslutning till Raspberry Pi **5**

6 Installation av nödvändiga bibliotek **5**
6.1 Installera WiringPi (GPIO-styrning) 5
6.2 Installera Mosquitto-klienter 5
6.3 Installera utvecklingsbibliotek 5
6.4 Verifiering via terminalen 5

7 Verifiering av koppling **6**
7.1 Testa LED (Utgång) 6
7.2 Testa din Knapp (Ingång) 6

8 Programkod **7**
8.1 Programkod: Sändare (publisher.c) 7
8.1.1 Terminal 1: Instruktioner för exekvering sändaren 7
8.2 Programkod: Mottagare (subscriber.c) 8
8.2.1 Terminal 2: Instruktioner för exekvering mottagaren 8
8.3 Flödesscheman för sändaren 9
8.4 Subscriber Flödesschema : Huvudloop och händelsehanterar 10
8.5 Koppling example / resultat 11

9 Slutsatser **12**

1 Sammanfattning

I denna laboration har vi undersökt hur man använder protokollet MQTT (Message Queuing Telemetry Transport) för att kommunicera mellan inbäddade system. Målet var att lära sig använda anpassade protokoll för publicering (publish) och prenumeration (subscribe). Vi har implementerat en lösning där en Raspberry Pi agerar både sändare och mottagare för att skicka sensordata eller statusmeddelanden via en central broker. Resultatet visar hur man genom att publicera på specifika topics"kan styra hårdvara, såsom lysdioder, baserat på värden från andra noder i nätverket.

2 MQTT: Teori

MQTT bygger på en Broker (medlare) som förmedlar meddelanden mellan sändare och mottagare.

- **Broker:** Server på IP `10.11.3.20`, Port `8080`.
- **Publish:** Sändaren skickar `on` eller `off` till en unik topic.
- **Subscribe:** Mottagaren lyssnar på samma topic och agerar på meddelandet.

Topics fungerar som adresser. Exempel på topics kan vara `test/temp1` eller unika ID:n som `grupp1/data`.

3 Hårdvarukrav

Hårdvarugrund för lösningen innefattar:

- En applikation som fungerar som sändare (publish) och skickar `on` när en tryckknapp aktiveras, annars `off`.
- En applikation som fungerar som prenumerant (subscribe) och tänder en lysdiod om meddelandet `on` tas emot.
  - **Tryckknapp:** Fungerar som sändare (publish) av statusen `on` eller `off`.
  - **Lysdiod (LED):** Den tänds när meddelandet `on` tas emot via MQTT.
  - **Resistor (10k-20kΩ):** Placeras i serie med knappen för pull-down ström. Vid 5v hålls strömen vid noll vid knappens viloläge.
  - **Optional: Resistor (220 Ω):** Placeras i serie med lysdioden för att begränsa strömmen och skydda GPIO-pinnarna. Vid 3,3V hålls strömstyrkan på en säker nivå kring 3,3 mA.
- Mjukvaror och stöd : För att labben ska fungera måaste både WiringPi (för GPIO) och Mosquitto (för MQTT) installeras.

4 Kretskopplings

För att genomföra laborationen på en och samma Raspberry Pi krävs en kretskoppling som hanterar både inmatning (knapp) och utmatning (LED) samtidigt via olika GPIO-pinnar.

4.1 Pin-konfiguration

Nedanstående bild visar hur komponenterna ska anslutas baserat på WiringPi-bibliotekets numrering och de fysiska numren på Raspberry Pi huvudkontakt (P1 Header).

BCM	WiringPi	Name	Physical		Name	WiringPi	BCM
		3.3v	1	2	5v		
2	8	SDA.1	3	4	5V		
3	9	SCL.1	5	6	0v		
4	7	1-Wire	7	8	TxD	15	14
		0v	9	10	RxD	16	15
17	0	GPIO. 0	11	12	GPIO. 1	1	18
27	2	GPIO. 2	13	14	0v		
22	3	GPIO. 3	15	16	GPIO. 4	4	23
		3.3v	17	18	GPIO. 5	5	24
10	12	MOSI	19	20	0v		
9	13	MISO	21	22	GPIO. 6	6	25
11	14	SCLK	23	24	CE0	10	8
		0v	25	26	CE1	11	7
0	30	SDA.0	27	28	SCL.0	31	1
5	21	GPIO.21	29	30	0v		
6	22	GPIO.22	31	32	GPIO.26	26	12
13	23	GPIO.23	33	34	0v		
19	24	GPIO.24	35	36	GPIO.27	27	16
26	25	GPIO.25	37	38	GPIO.28	28	20
		0v	39	40	GPIO.29	29	21
BCM	WiringPi	Name	Physical		Name	WiringPi	BCM

Figur 1: WiringPi-Fysiska kontakt

4.2 Mottagarkrets (LED)

Denna del styrs av `subscriber.c`.

- Identifiera **Physical Pin 1** (WiringPi 0/3.3v) och anslut till LED-lampans längre ben (anod).
- Anslut LED-lampans kortare ben (katod) till **Physical Pin 14** (GND).

4.3 Sändarkrets (Tryckknapp)

Denna del läses av `publisher.c`.

- Koppla ena sidan av tryckknappen(terminal 1a) till **Physical Pin 2** (5v).
- Koppla ena sidan av tryckknappen(terminal 2a) till **Physical Pin 12** (GPIO 1).
- Koppla ena sidan av tryckknappen(terminal 2b) till **10k/20k** Ω mellan **Physical Pin 6**(GND)

5 Fjärranslutning till Raspberry Pi

För att styra Raspberry Pi via nätverket används SSH

1. **Anslut till Wifi:** SSID: `dt171g`, Lösenord: `raspberrypi`
2. **Hitta IP:** Använd kommandot `nslookup dt171g-KITTNUMMER` i din dators terminal för att få fram IP-adressen
3. **Logga in:** Skriv `ssh pi@192.168.1.XXX`. Lösenordet är `raspberry`

6 Installation av nödvändiga bibliotek

6.1 Installera WiringPi (GPIO-styrning)

1. `sudo apt-get update && sudo apt-get upgrade`
2. `git clone https://github.com/WiringPi/WiringPi.git`
3. `cd WiringPi` och kör `./build`
4. Verifiera med `gpio -v`

6.2 Installera Mosquitto-klienter

Dessa verktyg används för att testa kommunikationen via terminalen:

- `sudo apt install mosquitto-clients`

6.3 Installera utvecklingsbibliotek

För att kunna kompilera C-kod med MQTT krävs följande:

- `sudo apt install libmosquitto-dev`

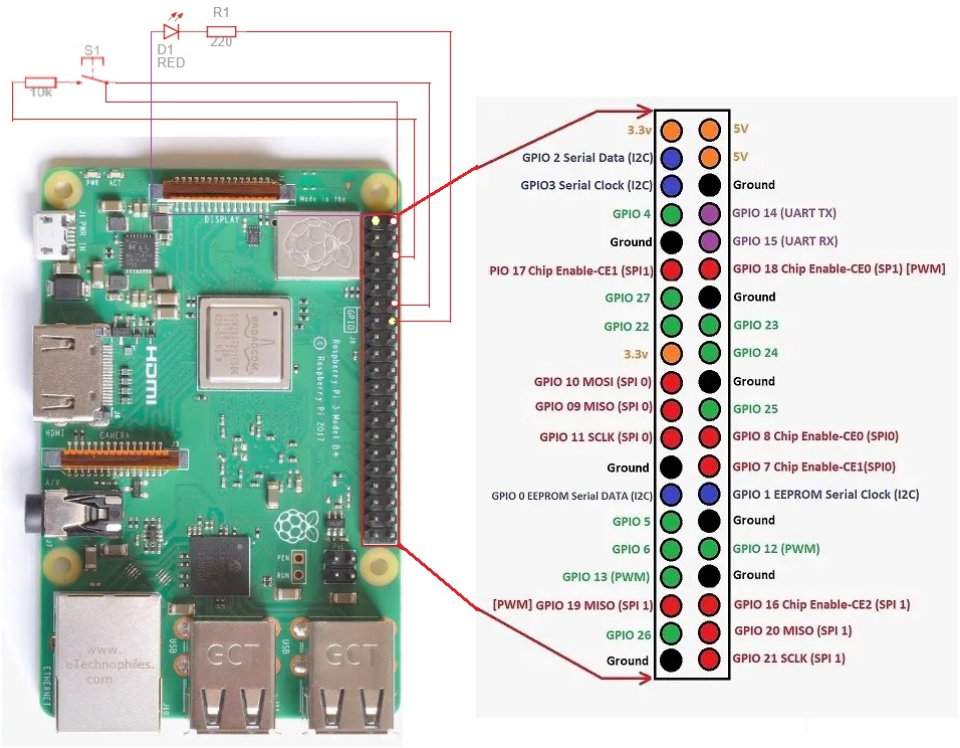
6.4 Verifiering via terminalen

Innan programmeringen påbörjas kan man testa anslutningen till brokern. Observera att ordning är viktigt:

- **Subscriber:** `mosquitto_sub -h 10.11.3.20 -p 8080 -t test/temp1`
- **Publisher:** `mosquitto_pub -h 10.11.3.20 -p 8080 -t test/temp1 -m "21.0C"`

7 Verifiering av koppling

Innan programmen startas kan kopplingen verifieras direkt via terminalen för att säkerställa att hårdvaran fungerar korrekt. Detta görs genom att styra och läsa av pinnarna manuellt utan att skriva ett helt program.



Figur 2: Färdig kopplings schema enligt punkt 4.2 och 4.3

7.1 Testa LED (Utgång)

Följande kommandon används för att kontrollera att lysdioden är korrekt ansluten:

- `gpio mode 0 out` : Ställer in WiringPi pinne 0 som en utgång.
- `gpio write 0 1` : Skickar ström (3,3V) till pinnen. Om allt är rätt kopplat ska din LED tändas nu.
- `gpio write 0 0` : (Valfritt) Stänger av strömmen och släcker lampan igen.

7.2 Testa din Knapp (Ingång)

Detta steg kontrollerar om pajen registrerar signalen från knappen på **Physical Pin 12** (WiringPi pinne 1):

- `gpio mode 1 in` : Ställer in WiringPi pinne 1 som en ingång.
- `gpio read 1` : Läser av den aktuella statusen på pinnen.
  - Om du **inte** trycker på knappen bör terminalen svara med **0**.
  - Om du **håller inne knappen** och kör kommandot igen bör terminalen svara med **1**.

8 Programkod

8.1 Programkod: Sändare (publisher.c)

Publisher (Sändare): Läser av hårdvaruinput (knapp) och skickar status till brokern. Denna fil hanterar knapptryckningar och publicerar ön"eller öff".

```
#include <mosquitto.h>
#include <stdio.h>
#include <wiringPi.h>
#include <unistd.h>

int main() {
  mosquitto_lib_init();
  struct mosquitto *mosq;
  int port = 8080;
  char ip[] = "10.11.3.20";
  int pingTimer = 50;
  char topic[] = "test/temp1";  // GIVE YOUR UNIQUE TOPIC FOR EXAMPLE "test/temp1"
  srandom(time(NULL));          // Give a random number generator seed
  //
  // Initialize mosquitto instance
  mosq = mosquitto_new("testnisse", NULL, NULL);  // GIVE YOUR UNIQUE ID FOR EXAMPLE
  "testnisse"
    //
    // Connect to broker
    mosquitto_connect(mosq, ip, port, pingTimer);



  while (true) {
    if(digitalRead(1) == HIGH)
    // Generate 20 random numbers and publish to broker
    for (size_t i = 0; i < 20; i++) {
      int num = random();  // Get random number
      char str[15];
      sprintf(str, "%i", num);                                          // Convert
          number to string
      mosquitto_publish(mosq, NULL, topic, sizeof(str), &str, 1, false);  //
      Publish to broker
        printf("%i\n", num);
      sleep(2);
    }
    return 0;
  }
}
```

8.1.1 Terminal 1: Instruktioner för exekvering sändaren

1. `gcc publisher.c -o pub -lwiringPi -lmosquitto`
2. `./pub`

8.2 Programkod: Mottagare (subscriber.c)

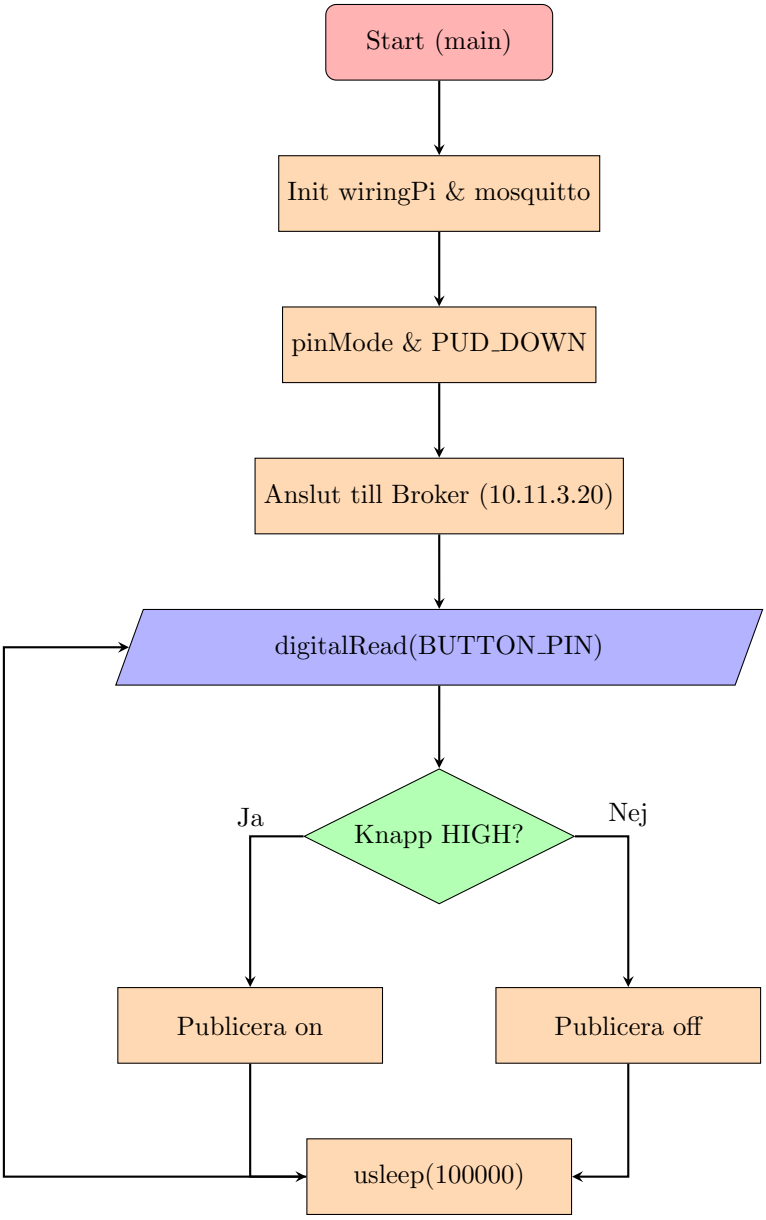
Subscriber (Mottagare): Lyssnar på brokern och styr hårdvaruoutput (LED) baserat på mottagen data. Denna fil prenumererar på topics och styr LED-lampan.

```
#include <mosquitto.h>
#include <stdio.h>
#include <string.h>
#include <wiringPi.h>

subscriber.c:

#include <mosquitto.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void message_callback(struct mosquitto mosq, voiduserdata, const struct mosquitto_message
     message) {
if (message->payloadlen) {
    printf(%s %s\n, message->topic, (char)message->payload);
    digitalWrite(0, HIGH);
    delay(500);
    digitalWrite(0, LOW);
    } else {
        printf(%s (null)\n, message->topic);
    }
}

int main() {
    mosquitto_lib_init();
    struct mosquitto mosq;
    charhost = "10.11.3.20";
    int port = 8080;
    int keepalive = 60;
    bool clean_session = true;
    mosq = mosquitto_new(NULL, clean_session, NULL);

    if (!mosq) {
        fprintf(stderr, Error: Out of memory.\n);
        return 1;
    }

    mosquitto_message_callback_set(mosq, message_callback);
    if (mosquitto_connect(mosq, host, port, keepalive)) {
        fprintf(stderr, Unable to connect.\n);
    return 1;
    }

    mosquitto_subscribe(mosq, NULL, "test/temp1", 0);  // GIVE YOUR UNIQUE TOPIC FOR
        EXAMPLE "test/temp1"
    mosquitto_loop_forever(mosq, -1, 1);
    mosquitto_destroy(mosq);
    mosquitto_lib_cleanup();
    digitalWrite(0, LOW)
    return 0;
}
```

8.2.1 Terminal 2: Instruktioner för exekvering mottagaren

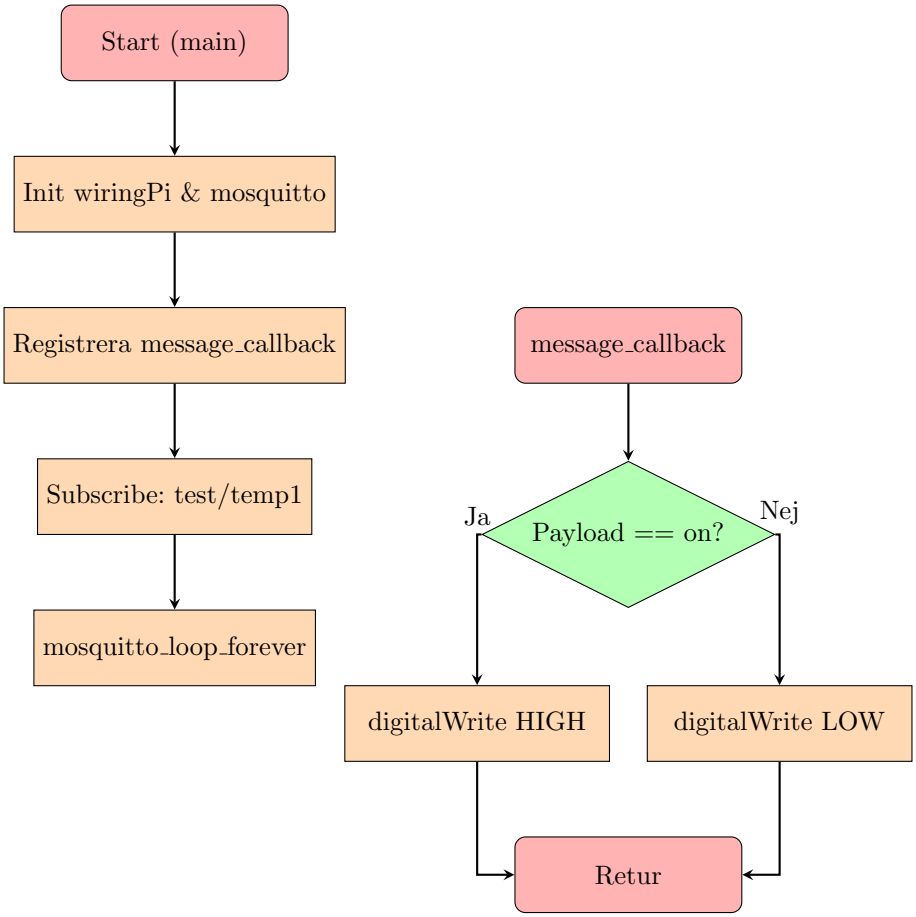
1. `gcc subscriber.c -o sub -lwiringPi -lmosquitto`
2. `./sub`

8.3 Flödesscheman för sändaren



Figur 3: Flödesschema för sändarens programlogik (Publisher).

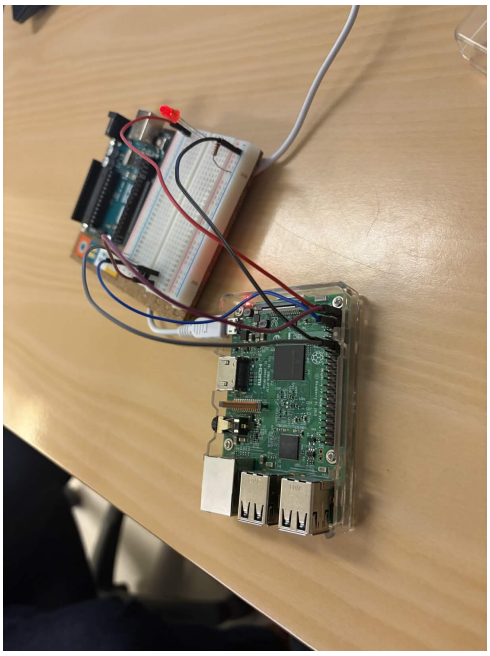
8.4 Subscriber Flödesschema : Huvudloop och händelsehanterar



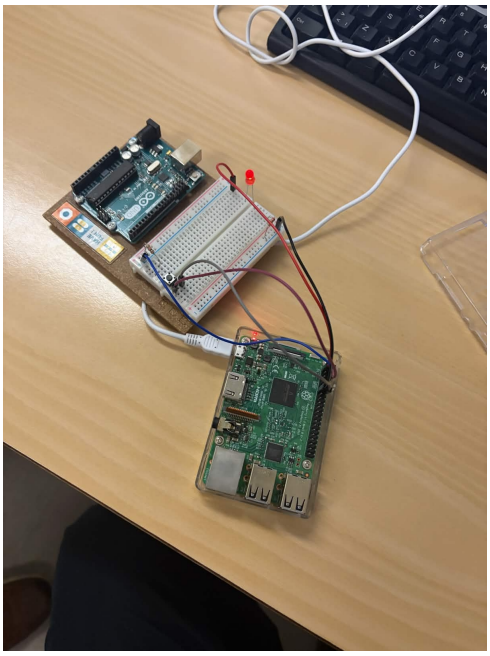
Figur 4: Flödesschema för mottagarens(Subscriber) huvudloop och händelsehanterare(Callback)

8.5 Koppling example / resultat

Utifrån vårt kodexempel har vi lyckats få till en fungerande anslutning med MQTT. När vi trycker på knappen startar sändarloopen som skickar iväg data till vår broker. Varje gång mottagaren tar emot ett meddelande via brokern så blinkar vår LED-lampa. Bilden här under visar hur den färdiga fysiska kopplingen ser ut.



Figur 5: Resultat : Vinkeln 1



Figur 6: Resultat : Vinkeln 2

9 Slutsatser

Laboration 4 har gett en djupare förståelse för hur man skapar distribuerade IoT-system med hjälp av MQTT. Genom att använda en broker blir systemet mycket mer flexibelt än vid direkt seriell kommunikation, då sändare och mottagare inte behöver känna till varandras IP-adresser. Galvanisk isolering, som diskuterades i Lab 3, är fortfarande relevant vid anslutning av sensorer, men MQTT möjliggör nu att dessa signaler kan spridas över ett helt nätverk på ett säkert och standardiserat sätt.